

Healthcare Resource Availability: Analyzing Global Disparities

ADAN8880 - Directed Practicum in Analytics

Professor: Nurtekin Savas

Date: April 26, 2025

Group Members: Jen Jiang, Yiwen Ma

Saving the Model for Deployment

After training, both the original and bias-mitigated XGBoost models were serialized and saved as .pkl files using Python's built-in pickle module.

Saving models as .pkl files enables:

- Rapid deployment without retraining every time.
- Consistency between the training and deployment environments.
- Version control if multiple model versions need to be maintained.

Sample code:

```
with open("xgb_model.pkl", "wb") as f:
    pickle.dump(xgb_model, f)
with open("xgb_mitigated_model.pkl", "wb") as f:
    pickle.dump(xgb_mitigated, f)
```

This approach ensures that models can be quickly loaded for batch scoring or integrated into production pipelines.

Documenting Environment Dependencies

Documenting the environment ensures reproducibility — so others (or future us) can re-run or deploy the model exactly as it was built.

Environment Details:

Operating System: macOS (presumed, should be confirmed).

- Python Version: 3.11
- Python Packages:
- pandas ($\geq 1.5.3$): For data handling and preprocessing.
- numpy ($\geq 1.24.2$): For numerical computations.
- scikit-learn ($\geq 1.2.2$): For model evaluation (mean_squared_error, r2_score).
- xgboost ($\geq 1.7.5$): For training XGBoost regressors.
- pickle: Standard library for model serialization.

Best Practice Tip: A requirements.txt file should also be generated with:

```
pip freeze > requirements.txt
```

Deployment Mode Selection: Batch Mode vs Real-Time Inference

Deployment Decision: Batch mode

Why Batch Mode?

- The model predicts life expectancy, which depends on macroeconomic and health indicators (like GNI per capita, health spending).
- These indicators are updated annually or quarterly, not in real-time.
- Real-time prediction would introduce unnecessary complexity and infrastructure costs without any real benefit.

Thus, it is most efficient to:

- Run the model periodically (e.g., once per year or quarter).
- Update predictions only when new data becomes available.

Batch mode ensures cost-efficiency, simplicity, and alignment with the natural data update cycle.

Performance Metrics and Monitoring Plan

Monitoring model performance is critical for early detection of degradation or failures in production.

Metrics to Track:

- **Model Metrics:**
 - Mean Squared Error (MSE): Measures average squared difference between predicted and actual values.
 - R^2 Score: Represents how well the model explains variance in the target variable.
- **Business Metrics:**
 - Prediction Drift: Detects if the model's average predictions significantly shift over time, even when inputs have not.
- **Operational Metrics:**
 - Model Load Time: Measures how fast the saved model loads into memory, critical for batch job efficiency.

Why These Metrics Matter:

- MSE and R^2 ensure the model remains accurate and relevant.
- Prediction drift flags hidden issues like changing data distributions.
- Load time ensures deployment operations remain smooth and efficient.

Thresholds for Green/Yellow/Red Alerts

We define clear thresholds to determine the system's health and whether intervention is needed:

Metric	Green (Normal)	Yellow (Warning)	Red (Critical)
MSE	≤ 10	$10 < \text{MSE} \leq 20$	> 20
R^2 Score	≥ 0.75	$0.65 \leq R^2 < 0.75$	< 0.65
Prediction Drift	$\leq \pm 5\%$ from historical average	$\pm 5\% - 10\%$ drift	$> \pm 10\%$ drift
Model Load Time	≤ 2 seconds	2–5 seconds	> 5 seconds

- Green: System is healthy, no immediate action needed.
- Yellow: Caution zone. Investigate proactively.
- Red: Critical. Immediate rollback or retraining required.

Risk Mitigation Strategies

Having a mitigation plan ensures quick recovery and reduces downtime:

- Green Flags:
 - Continue normal monitoring.
 - Log metrics for historical analysis.
- Yellow Flags:
 - Conduct root-cause analysis:
 - Is the feature distribution changing?
 - Is the model becoming less predictive?
 - Compare performance with a backup model if available.
 - Schedule early retraining if needed.
- Red Flags:
 - Immediate rollback to a previous working model.
 - Notify stakeholders (business, tech teams).
 - Launch incident review to diagnose underlying causes.
 - Rebuild or retrain the model using updated data.

Retraining Strategy

Planned Retraining Frequency:

Every 6 to 12 months (annual schedule recommended).

Triggers for Early Retraining:

- Yellow or red alerts on MSE or R^2 .
- Significant prediction drift detected.
- Major shifts in underlying data (e.g., a pandemic affecting life expectancy trends).

Why This Schedule?

- Life expectancy predictors change slowly over time.
- Annual retraining keeps the model reasonably current without wasting resources.

Data Drift and Concept Drift Mitigation

Both types of drift can impact the model's long-term viability.

- Data Drift (Input Features Shift):
 - Example: GNI per capita distributions change significantly.
 - Detection:
 - Monitor distributions of input features monthly.
 - Use statistical drift tests like KS-Test.
 - Mitigation:
 - Preprocessing updates.
 - Retraining with updated data.
- Concept Drift (Relationship Between Features and Target Changes):
 - Example: Health expenditure impact on life expectancy changes after a major health policy reform.
 - Detection:
 - Monitor model performance (MSE, R^2).
 - Watch for sustained drops.
 - Mitigation:
 - Retrain models.
 - Revisit feature selection or model type if needed.

Best Practice: Set up automated drift detection in the production pipeline to trigger retraining or human review automatically when needed.